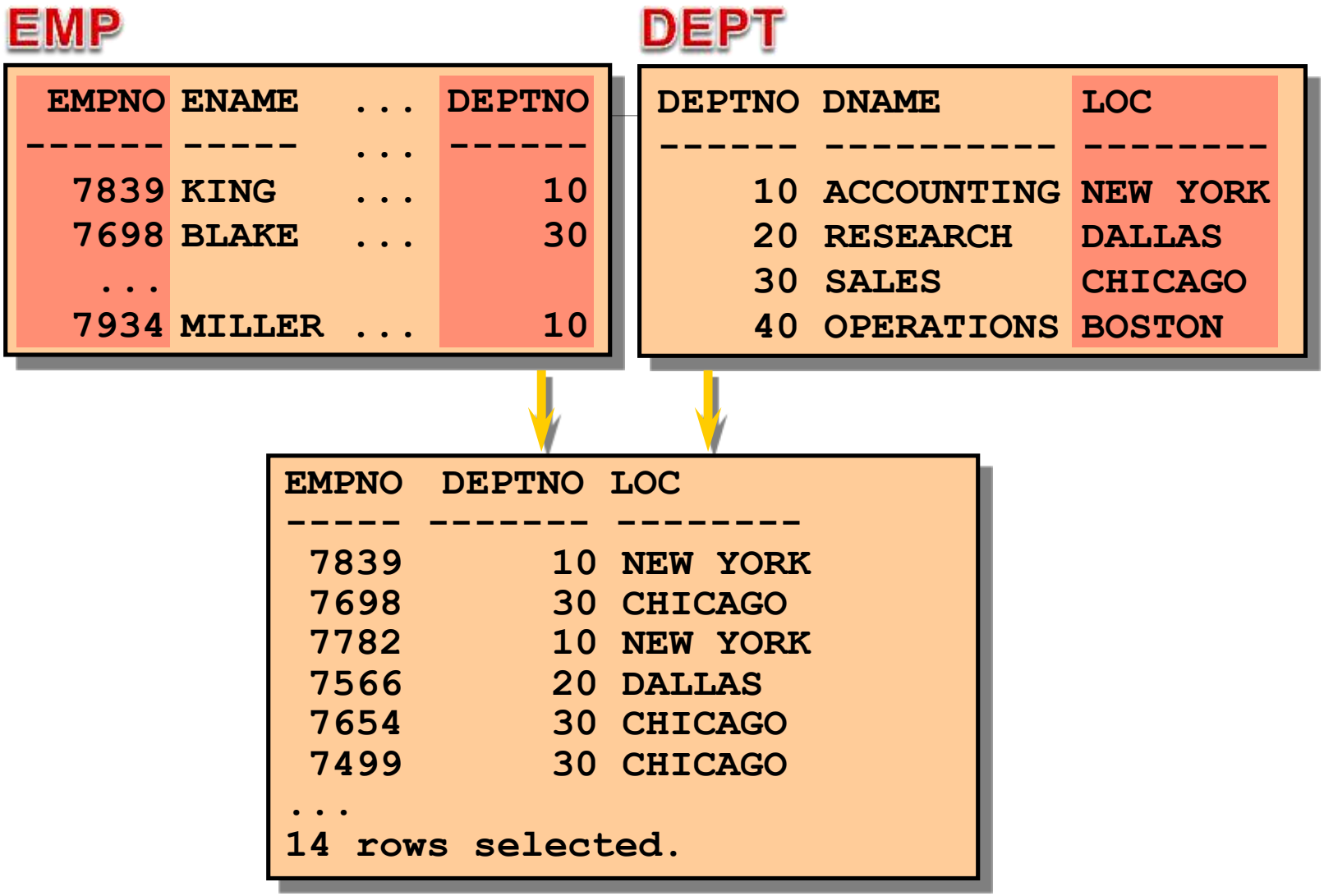
Displaying Data from Multiple Tables

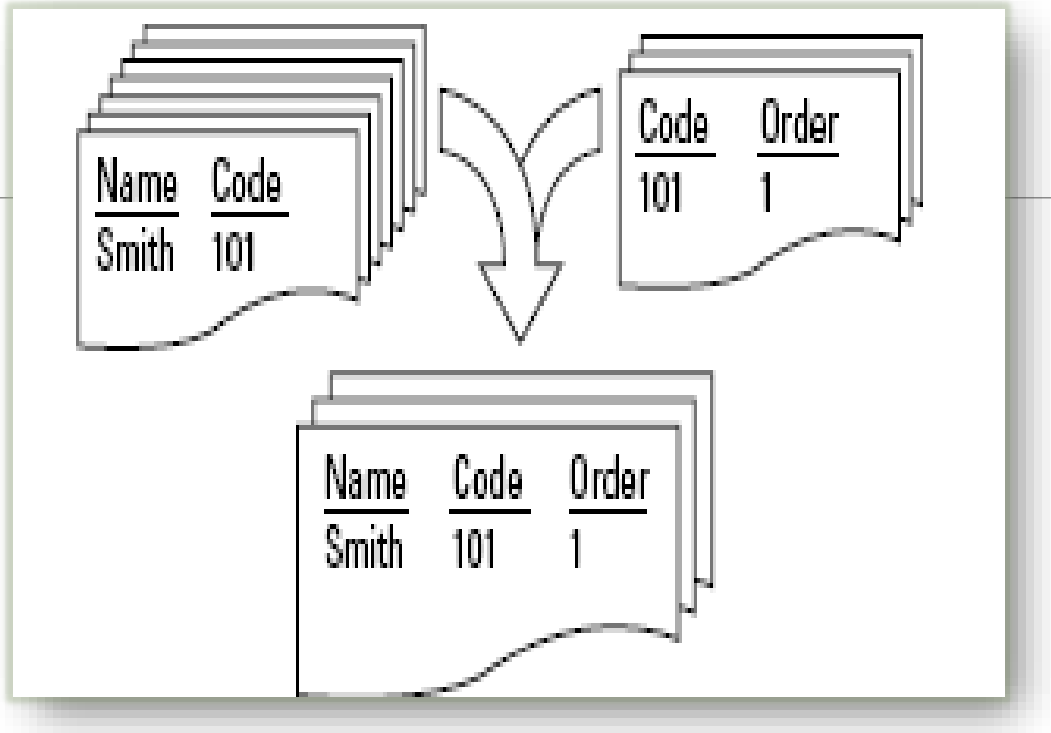
Objectives

- After completing this lesson, you should be able to do the following:
  - **Write SELECT statements to access data from more than one table using equality and nonequality joins**
  - **View data that generally does not meet a join condition by using outer joins**
  - **Join a table to itself**

Obtaining Data from Multiple Tables



What Is a Join?



- A join merges rows from one data set with rows from another data set, creating a new set of rows that includes columns from both.

What Is a Join?

Use a join to query data from more than one table

Rows in one table can be joined to rows in another table according to common values existing in corresponding columns, that is, usually primary and foreign key columns

Guidelines in Using a Join

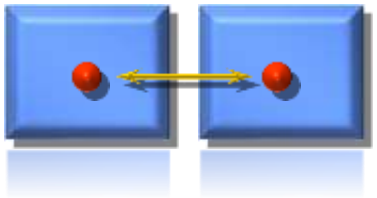
- **When writing a SELECT statement that joins tables, precede the column name with the table name for clarity and to enhance database access**

- **If the same column name appears in more than one table, the column name must be prefixed with the table name**

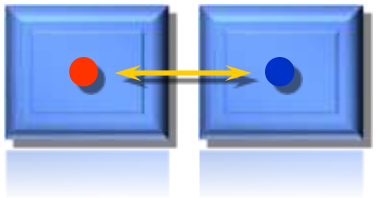
- **To join *n* tables together, you need a minimum of (*n-1*) join conditions. Therefore, to join four tables, a minimum of three joins are required**

Types of Joins

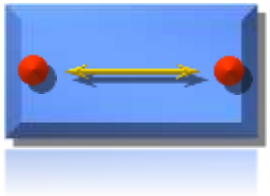
Equijoin



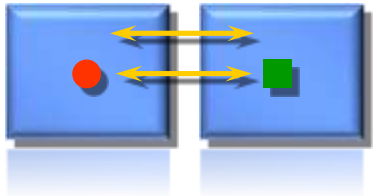
Outer join



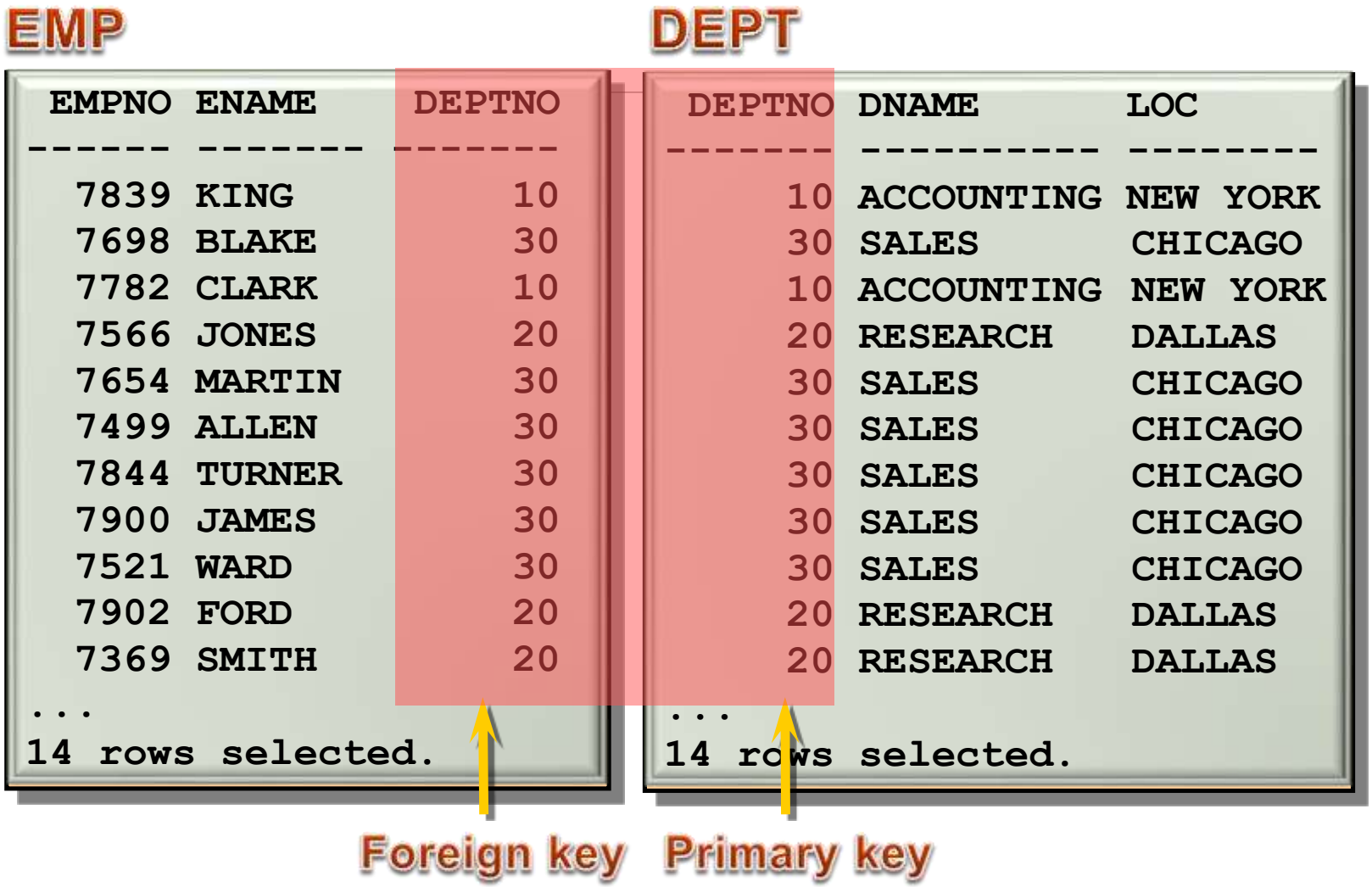
Self join



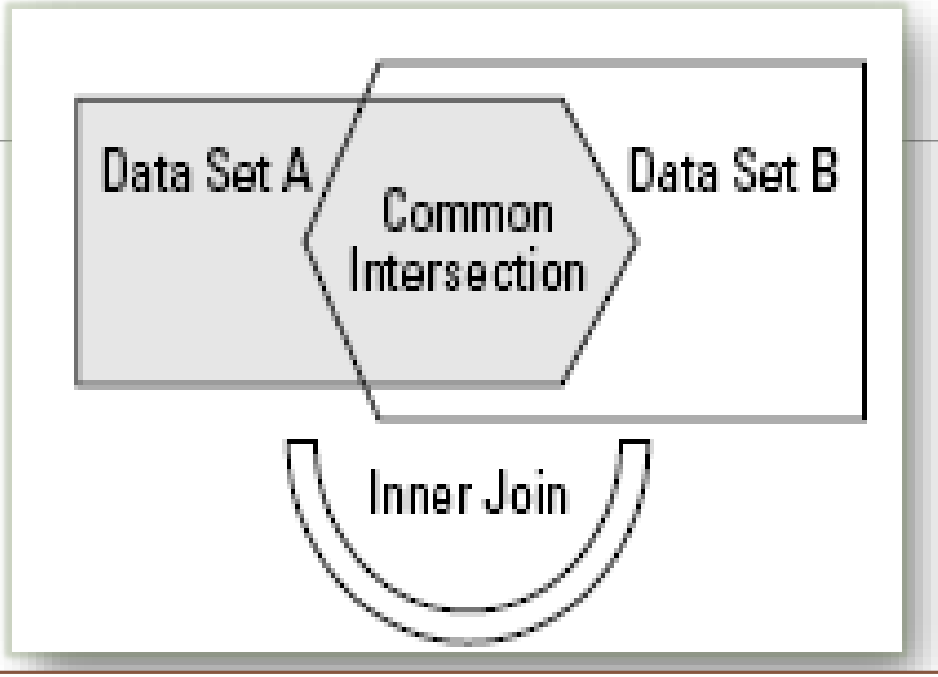
Cross join



What Is an Equijoin?



What Is an Equijoin?



- **The inner join includes only those rows from each side of the join that are contained within the intersection of the two data sources.**

What Is an Equijoin?

- Equijoins are also called *inner joins*

- It returns all the columns in both tables, and returns only the rows for which there is an equal value in the join column.

What Is a Join?

```
SELECT table1.col, table2.col
FROM   table1
[INNER] JOIN table2
        [ON table1.col = table2.col]
```

table1.column - denotes the table and column from which data is retrieved

JOIN **- identifies the second table**

ON **– defines the relation between two the tables**

table1.column1 = table2.column2

- is the condition that joins (or relates) the tables together

Retrieving Records with Equijoins

```
SELECT Products.ProductID,
   Products.ProductName, Products.CategoryID,
   Categories.CategoryID, Categories.CategoryName
FROM  Products.Products
INNER JOIN Products.Categories
  ON Products .CategoryID = Categories .CategoryID
```

```
ProductID  ProductName     CategoryId CategoryId CategoryName
---------- ---------------  ---------- ---------- ---------------
1          Product HHYDP           1          1 Beverages
2          Product RECZE           1          1 Beverages
3          Product IMEHJ           2          2 Condiments
...
77 rows selected.
```

Qualifying Ambiguous Column Names

- Use table prefixes to qualify column names that are in multiple tables.

- **Improve performance by using table prefixes.**
- **Distinguish columns that have identical names but reside in different tables by using column aliases.**

Using Table Aliases

- **Use table *aliases* instead of table names particularly if table names are lengthy**

- **Just as a column alias gives a column another name, a table alias gives a table another name**

- **If a table alias is used for a particular table name in the FROM clause, then that table alias must be substituted for the table name throughout the SELECT statement**

Using Table Aliases

```
SELECT P.ProductID, P.ProductName,
       P.CategoryID, C.CategoryID,
       C.CategoryName
FROM  Production.Products P
INNER JOIN Production.Categories C
  ON P.CategoryID = C.CategoryID
```

ProductID	ProductName	CategoryId	CategoryId	CategoryName
1	Product HHYDP	1	1	Beverages
2	Product RECZE	1	1	Beverages
3	Product IMEHJ	2	2	Condiments

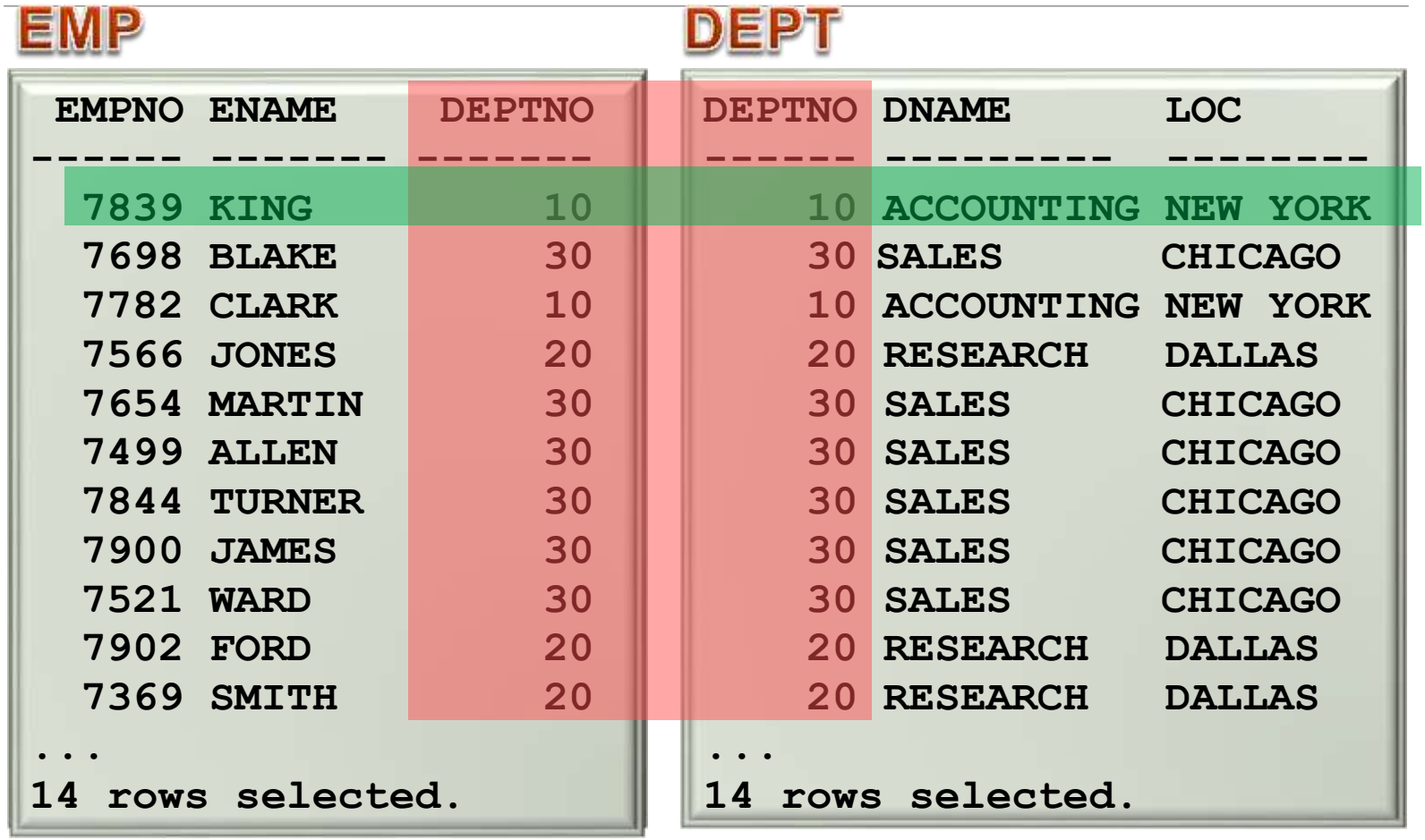
...

77 rows selected.

```
SELECT P.ProductID, P.ProductName,
       P.CategoryID, C.CategoryID,
       C.CategoryName
FROM  Production.Products P
INNER JOIN Production.Categories C
   ON P.CategoryID = C.CategoryID
WHERE P. CategoryID = 1
```

```
ProductID  ProductName  CategoryId CategoryId CategoryName
---------- ------------- ---------- ---------- ---------------
1          Product HHYDP 1          1          Beverages
2          Product RECZE 1          1          Beverages
24         Product QOGNU 1          1          Beverages
...
12 rows selected.
```

Additional Search Conditions Using the AND Operator



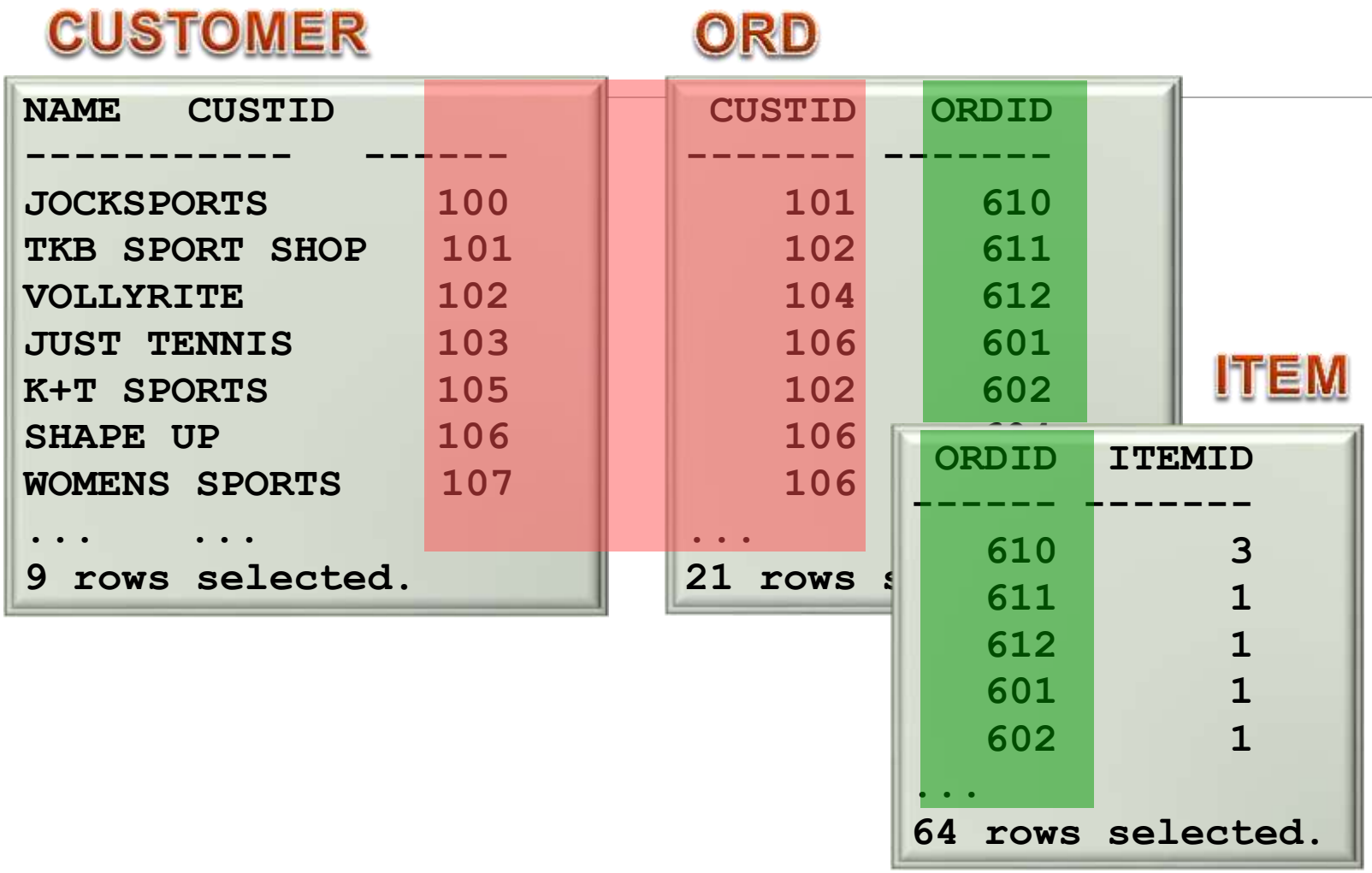
Additional Search Conditions Using the AND Operator

```
SELECT P.ProductID, S.SupplierID,
       P.ProductName, S.CompanyName
FROM  Production.Products P
INNER JOIN Production.Suppliers S
  ON P.SupplierID = S.SupplierID
   AND P. SupplierID = 1
```

ProductID	SupplierID	ProductName	CompanyName
1	1	Product HHYDP	Supplier SWRXU
2	1	Product RECZE	Supplier SWRXU
3	1	Product IMEHJ	Supplier SWRXU

3 rows selected.

Joining More Than Two Tables



```
SELECT P.ProductID, P.ProductName,
       P.CategoryID, C.CategoryName,
       P.SupplierID, S.CompanyName
FROM  Production.Products P
INNER JOIN Production.Categories C
  ON P.CategoryID = C.CategoryID
INNER JOIN Production.Suppliers  S
  ON S. SupplierID = P.SupplierID
```

```
ProductID  ProductName  CategoryId CategoryName  SupplierID  CompanyName
---------- ------------- ---------- ------------- ----------- --------------------
1          Product HHYDP 1          Beverages     1           Supplier SWRXU
2          Product RECZE 1          Beverages     1           Supplier SWRXU
3          Product IMEHJ 2          Condiments    1           Supplier SWRXU
...
77 rows selected.
```

Joining More Than Two Tables

"Who are the suppliers and what are the products of the category Dairy Products"

1. The Suppliers for the "Supplier Name"
2. The Product table for the "Products"
3. The Categories table for the "Dairy Products"

Joining More Than Two Tables

```
SELECT P.ProductName, C.CategoryName,
            S.CompanyName Supplier
FROM  Production.Products P
INNER JOIN Production.Categories C
   ON P.CategoryID = C.CategoryID
INNER JOIN Production.Suppliers  S
   ON S. SupplierID = P.SupplierID
WHERE c.categoryid = 4
```

```
ProductName          CategoryName        Supplier
 ---------------     ------------        --------------------
Product QMVUN        Dairy Products      Supplier EQPNC
Product OSFNS        Dairy Products      Supplier EQPNC
Product XWOXC        Dairy Products      Supplier KEREV
...
10 rows selected.
```

Joining More Than Two Tables

"What are the products (including total quantity and total sales) ordered on the year 2006"

1. The Orders table for the "purchased"
2. The [Order Detail] table for the "purchased"
3. The Product table for the "Products"



```
SELECT P.ProductName,
       SUM(OD.Qty) Total_Qty, OD.UnitPrice,
       OD.UnitPrice * SUM(OD.Qty)  Total_Sales
FROM Sales.OrderDetails OD
INNER JOIN Sales.ORDERS O
    ON O.OrderID = OD.OrderID
   AND YEAR(O.OrderDate) = 2006
INNER JOIN Production. PRODUCTS P
   ON P.ProductID = OD.ProductID
GROUP BY P.ProductName, OD.UnitPrice
ORDER BY P.ProductName
```

Joining More Than Two Tables

"Who purchased Beverages?"

1. The Customer table for the "who"
2. The Orders table for the "purchased"
3. The [Order Details] table for the "purchased"
4. The Product table for the "Beverages"
5. The ProductCategory table for the "Beverages"



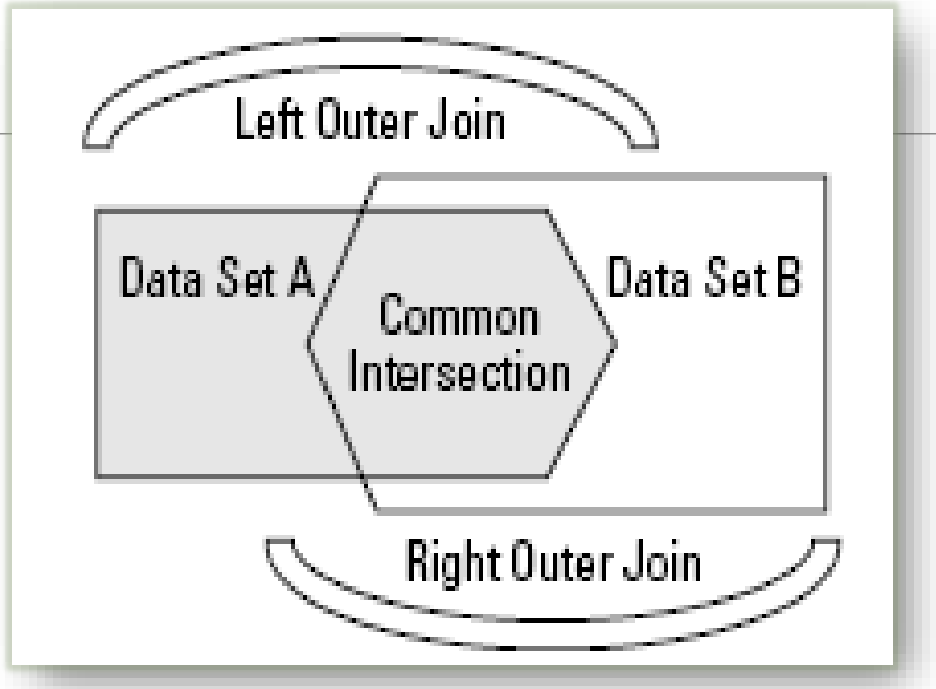
Joining More Than Two Tables

```
SELECT DISTINCT CU.CompanyName,
        P.ProductName, C.CategoryName
FROM Sales.OrderDetails OD
INNER JOIN Sales.ORDERS O
        ON O.OrderID = OD.OrderID
INNER JOIN Production.PRODUCTS P
        ON P.ProductID = OD.ProductID
INNER JOIN Production.CATEGORIES C
        ON P.categoryID = C.CategoryID
        AND C.CategoryName = 'Beverages'
INNER JOIN Sales.CUSTOMERS CU
        ON CU.CustID = O.CustID
ORDER BY CU.CompanyName
```

Outer Joins


DISCOUNT
STORES
DISCOUNTTYPE STOR_ID
Initial Customer NULL
Volume Discount NULL
Customer Discount 8042
STOR_ID STOR_NAME
6380 Eric the Read Books
7066 Barnum's
7067 News & Brews
7131 Doc-U-Mat:Quality
7896 Fricative Bookshop
8042 Bookbeat...
No Stor_ID for other DiscountType

Outer Joins



- **An outer join includes not only rows from the two data sources with a match but also unmatched rows from outside the intersection.**

Outer Joins

- An outer join extends the inner join by adding the nonmatching data from the left or right data set.
- Some of the data in the result set produced by an outer join will look just like the data from an inner join.
- Any rows from the outer-join table that do not have a match in the other side of the join will return data only from the outer-join table. In this case, columns from the other data source will have null values.

Outer Joins

- Outer joins are often employed when a secondary table has a foreign-key constraint to the primary table and permits nulls in the foreign key

Outer Joins

```
SELECT    select_list
FROM <table you want to be the LEFT table>
<LEFT | RIGHT OUTER JOIN >
          <table you want to be the RIGHT table>
[ON join_condition]
```

Outer Join Examples

All customers with order details if present:

```
SELECT c.custid, c.contactname,
    o.orderid, o.orderdate
FROM Sales.Customers AS C
LEFT OUTER JOIN Sales.Orders AS O
    ON c.custid = o.custid;
```

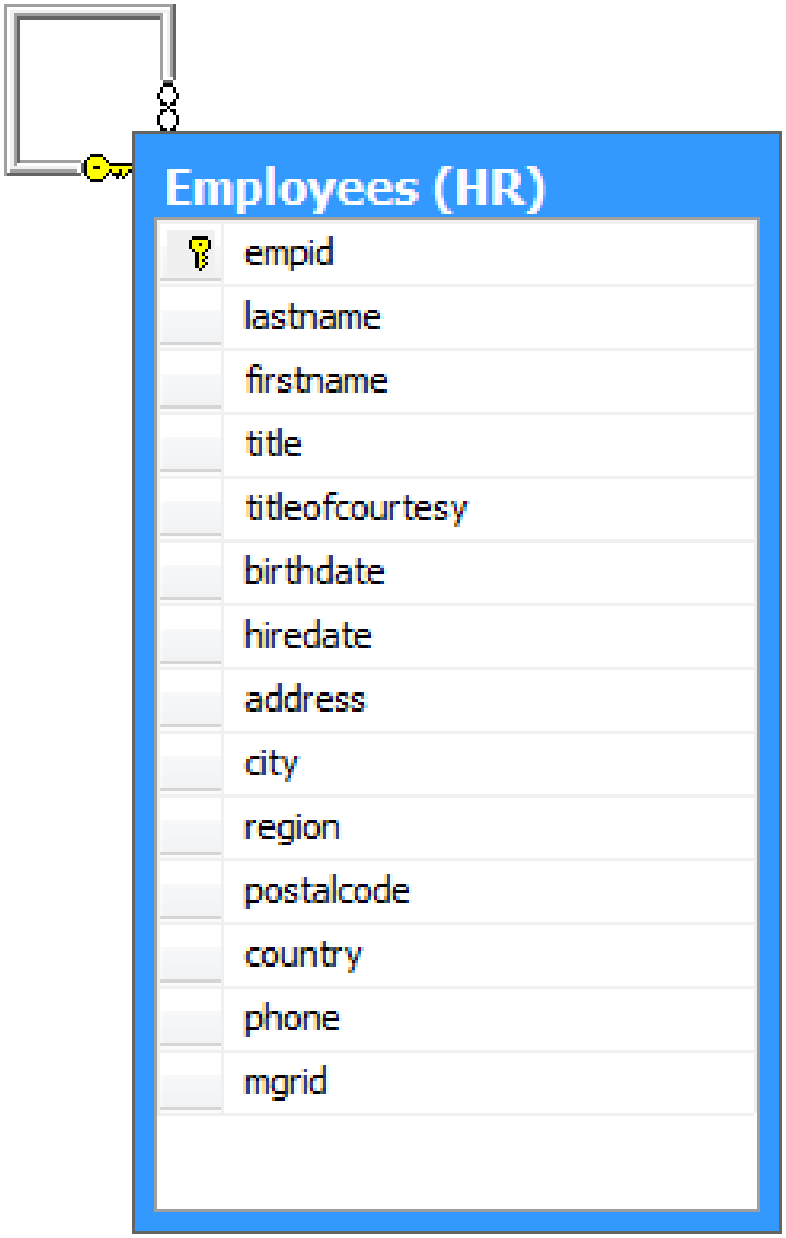
Customers that did not place orders:

```
SELECT c.custid, c.contactname,
    o.orderid, o.orderdate
FROM Sales.Customers AS C
LEFT OUTER JOIN Sales.Orders AS O
    ON c.custid = o.custid
WHERE o.orderid IS NULL;
```

Self Joins

- A *self-join* is a join that refers back to the same table

Self Joins



To find the name of each employee's manager, you need to join the EMPLOYEES table to itself, or perform a self join

Self Joins

```
SELECT E.EmpID,
   E.FirstName+ ' '+E.LastName EmpName,   E.MgrID,
   M.FirstName+ ' '+M.LastName ManagerName
FROM HR.EMPLOYEES E
INNER JOIN HR.EMPLOYEES M
   ON E.MgrId = M.EmpID
```

EmployeeID	EmpName	MgrID	ManagerName
2	Don Funk	1	Sara Davis
3	Judy Lew	2	Don Funk
4	Yael Peled	3	Judy Lew

…

8 rows selected.

Cross Join

- The cross join, also called an unrestricted join, is a pure relational algebra multiplication of the two source tables.
- Without a join condition restricting the result set, the result set includes every possible combination of rows from the data
- Each row in data set one is matched with every row in data set two
- This type of result set is referred to as a *Cartesian product*.



Cartesian Product

A Cartesian product is formed when:

- A join condition is omitted
- A join condition is invalid
- All rows in the first table are joined to all rows in the second table

Cartesian Product

To avoid a Cartesian product, always include a valid join condition

Generating a Cartesian Product

EMP (14 rows)

EMPNO	ENAME	...	DEPTNO
7839	KING	...	10
7698	BLAKE	...	30
...			
7934	MILLER	...	10

DEPT (4 rows)

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

"Cartesian product: 14*4=56 rows"

ENAME	DNAME
KING	ACCOUNTING
BLAKE	ACCOUNTING
...	
KING	RESEARCH
BLAKE	RESEARCH
...	

56 rows selected.